

3F8: Inference: Short Lab Report

Zihe Liu

February 26, 2026

Abstract

This is the abstract.

Try for 1-2 sentences on each of: motive (what it's about), method (what was done), key results and conclusions (the main outcomes).

- Don't exceed 3 sentences on any one.
- Write this last, when you know what it should say!

Introduction

1. What is the problem and why is it interesting?
2. What novel follow-up will the rest of your report present?

Exercise a): Gradients of Log-Likelihood

In this exercise we consider the logistic classification model (aka logistic regression) and derive the gradient of the log-likelihood with respect to the parameter vector β , given a vector of binary labels $\mathbf{y}$ and a matrix of input features $\mathbf{X}$. The probability of a positive class label is given by the logistic (sigmoid) function $\sigma(a) = 1/(1 + e^{-a})$ applied to the activations:

$$P(y^{(n)} = 1 \mid \tilde{\mathbf{x}}^{(n)}) = \sigma(\beta^T \tilde{\mathbf{x}}^{(n)}), \quad P(y^{(n)} = 0 \mid \tilde{\mathbf{x}}^{(n)}) = 1 - \sigma(\beta^T \tilde{\mathbf{x}}^{(n)}) = \sigma(-\beta^T \tilde{\mathbf{x}}^{(n)}),$$

where $\tilde{\mathbf{x}}^{(n)} = (1, \mathbf{x}^{(n)})^T$ are the bias-augmented inputs, so that $\beta^T \tilde{\mathbf{x}}^{(n)} = \beta_0 + \sum_{d=1}^D \beta_d x_d^{(n)}$. These two cases can be written compactly as a single Bernoulli likelihood:

$$P(y^{(n)} \mid \tilde{\mathbf{x}}^{(n)}, \beta) = \sigma(\beta^T \tilde{\mathbf{x}}^{(n)})^{y^{(n)}} \left(1 - \sigma(\beta^T \tilde{\mathbf{x}}^{(n)})\right)^{1-y^{(n)}}$$

Assuming N datapoints are independent and identically distributed, the likelihood of the full dataset is:

$$P(\mathbf{y} \mid \mathbf{X}, \beta) = \prod_{n=1}^N \sigma(\beta^T \tilde{\mathbf{x}}^{(n)})^{y^{(n)}} \left(1 - \sigma(\beta^T \tilde{\mathbf{x}}^{(n)})\right)^{1-y^{(n)}}$$

Giving the log-likelihood $\mathcal{L}(\beta) = \log P(\mathbf{y} \mid \mathbf{X}, \beta)$ of:

$$\mathcal{L}(\beta) = \sum_{n=1}^N \left[y^{(n)} \log \sigma(\beta^T \tilde{\mathbf{x}}^{(n)}) + (1 - y^{(n)}) \log \left(1 - \sigma(\beta^T \tilde{\mathbf{x}}^{(n)})\right) \right]$$

Note the derivative of the logistic function:

$$\frac{d\sigma(a)}{da} = \frac{e^{-a}}{(1 + e^{-a})^2} = \frac{1}{1 + e^{-a}} \cdot \frac{e^{-a}}{1 + e^{-a}} = \sigma(a) (1 - \sigma(a))$$

Let $a^{(n)} = \beta^T \tilde{\mathbf{x}}^{(n)}$, so that $\frac{\partial a^{(n)}}{\partial \beta} = \tilde{\mathbf{x}}^{(n)}$. Then our derivatives are

$$\frac{\partial}{\partial \beta} \left[y^{(n)} \log \sigma(a^{(n)}) \right] = y^{(n)} \cdot \frac{1}{\sigma(a^{(n)})} \cdot \underbrace{\frac{d\sigma}{da}}_{\sigma(1-\sigma)} \cdot \underbrace{\frac{\partial a^{(n)}}{\partial \beta}}_{\tilde{\mathbf{x}}^{(n)}} = y^{(n)} (1 - \sigma(a^{(n)})) \tilde{\mathbf{x}}^{(n)}$$

$$\frac{\partial}{\partial \beta} \left[(1 - y^{(n)}) \log(1 - \sigma(a^{(n)})) \right] = (1 - y^{(n)}) \cdot \frac{-\sigma(a^{(n)})(1 - \sigma(a^{(n)}))}{1 - \sigma(a^{(n)})} \cdot \tilde{\mathbf{x}}^{(n)} = -(1 - y^{(n)}) \sigma(a^{(n)}) \tilde{\mathbf{x}}^{(n)}$$

Combining these two contributions for datapoint n :

$$y^{(n)}(1 - \sigma(a^{(n)})) \tilde{\mathbf{x}}^{(n)} - (1 - y^{(n)}) \sigma(a^{(n)}) \tilde{\mathbf{x}}^{(n)} = (y^{(n)} - \sigma(a^{(n)})) \tilde{\mathbf{x}}^{(n)}.$$

Summing over all N datapoints,

$$\boxed{\frac{\partial \mathcal{L}(\beta)}{\partial \beta} = \sum_{n=1}^N (y^{(n)} - \sigma(\beta^T \tilde{\mathbf{x}}^{(n)})) \tilde{\mathbf{x}}^{(n)} = \tilde{\mathbf{X}}^T (\mathbf{y} - \boldsymbol{\sigma}),}$$

where $\tilde{\mathbf{X}}$ is the $N \times (D+1)$ matrix whose n -th row is $(\tilde{\mathbf{x}}^{(n)})^T$, and $\boldsymbol{\sigma}$ is the N -dimensional vector with entries $\sigma(\beta^T \tilde{\mathbf{x}}^{(n)})$. The vectorised form follows because the sum $\sum_n (\cdot) \tilde{\mathbf{x}}^{(n)}$ is equivalent to the matrix product $\tilde{\mathbf{X}}^T$ applied to the residual vector $(\mathbf{y} - \boldsymbol{\sigma})$.

Exercise b) Vectorized Gradient Ascent

In this exercise we write vectorised pseudocode to estimate the parameters β using gradient ascent of the log-likelihood. The update rule is

$$\beta^{(\text{new})} = \beta^{(\text{old})} + \eta \left. \frac{\partial \mathcal{L}(\beta)}{\partial \beta} \right|_{\beta=\beta^{(\text{old})}} = \beta^{(\text{old})} + \eta \tilde{\mathbf{X}}^T (\mathbf{y} - \boldsymbol{\sigma}^{(\text{old})}),$$

where $\boldsymbol{\sigma}^{(\text{old})}$ is the vector of predictions $\sigma(\beta^{(\text{old})T} \tilde{\mathbf{x}}^{(n)})$ under the current parameters. The pseudocode is as follows:

Function `estimate_parameters`:

Input: feature matrix $\mathbf{X}$ ($N \times D$), labels $\mathbf{y}$ ($N \times 1$),
learning rate η , number of iterations T
Output: vector of coefficients $\mathbf{b}$ ($(D+1) \times 1$)

Code:

```
X_tilde = [ones(N,1), X]          % augment with bias column
b = randn(D+1, 1)                 % initialise randomly

for t = 1 to T:
    sigma = 1 / (1 + exp(-X_tilde * b)) % predictions (N x 1)
    grad = X_tilde^T * (y - sigma)      % gradient ((D+1) x 1)
    b = b + eta * grad                  % gradient ascent step

return b
```

The learning rate η should be chosen small enough that the log-likelihood increases roughly monotonically, but large enough that convergence is achieved within a reasonable number of iterations. In practice, one can start with a small value (e.g. $\eta = 0.01$) and increase it until the learning curve begins to oscillate, then reduce slightly. The number of iterations T should be large enough for the log-likelihood to plateau.

Exercise c) Data Visualisation

The dataset consists of $N = 1000$ datapoints with two-dimensional input features $\mathbf{x}^{(n)} \in \mathbb{R}^2$ and binary class labels $y^{(n)} \in \{0, 1\}$. The dataset is visualised in Figure 1.

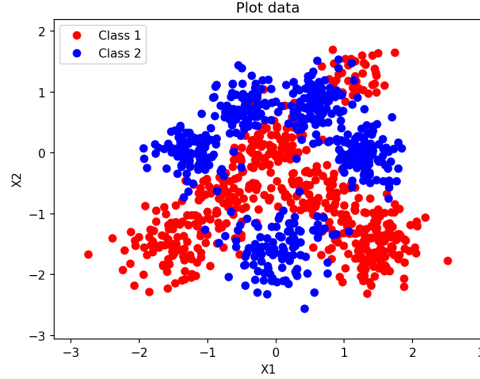


Figure 1: Visualisation of the dataset in the (X_1, X_2) input space. Red points are Class 1 ($y = 0$) and blue points are Class 2 ($y = 1$).

From Figure 1, we observe that Class 2 (blue) is concentrated in the upper and lower area of the input space, while Class 1 (red) is interspersed between the 2 blue regions. The two classes overlap significantly, particularly around the boundaries, and no single straight line can cleanly separate them. A linear classifier produces a decision boundary of the form $\beta^T \tilde{\mathbf{x}} = 0$, which is a straight line in 2D. Given the non-linear arrangement of the classes, such a linear boundary would inevitably misclassify a substantial portion of the data. A non-linear decision boundary would be needed to capture the true class structure more accurately.

Exercise d) Training the Linear Classifier

The data is split randomly¹ into training ($N_{\text{train}} = 800$) and test ($N_{\text{test}} = 200$) sets. Gradient ascent is implemented as:

```
sigmoid_value = predict(X_tilde_train, w)
w = w + alpha * np.dot(X_tilde_train.T, y_train - sigmoid_value)
```

We train the classifier with learning rate $\eta = 0.001$ for $T = 100$ iterations. The average log-likelihood on the training and test sets as the optimisation proceeds is shown in Figure 2.

Both curves increase monotonically and plateau after approximately 20 iterations, indicating that the optimisation has converged and $\eta = 0.001$ is an appropriate learning rate. The training and test curves follow a similar trajectory, suggesting there is no overfitting – as expected for a model with only $D + 1 = 3$ parameters fitting 800 datapoints.

Figure 3 displays the contours of the predictive probabilities overlaid on the data. The contours are parallel straight lines, confirming the linear nature of the decision boundary. The 0.5 contour (the decision boundary) runs roughly diagonally, separating the upper region (higher $P(y = 1)$) from the lower region. However, as anticipated in Exercise c), a linear boundary cannot capture the non-linear class structure: many red (Class 1) and blue (Class 2) points lie on the wrong side of the decision boundary. This motivates the non-linear feature expansion in later exercises.

¹A fixed random seed of 1 was used for reproducibility.

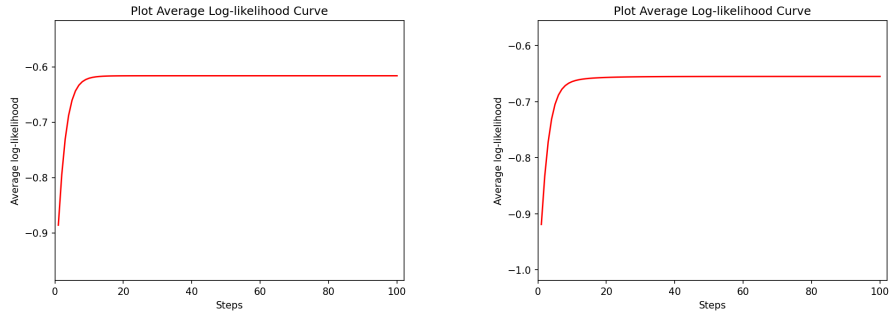


Figure 2: Learning curves showing the average log-likelihood per datapoint on the training (left) and test (right) datasets.

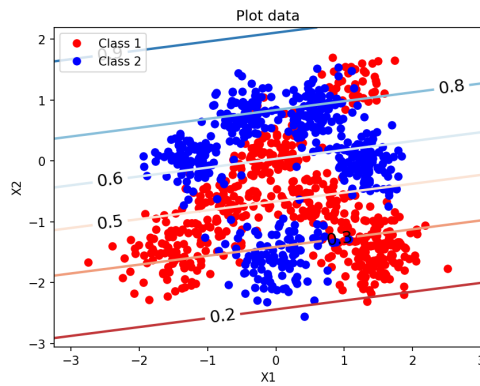


Figure 3: Contours of the class predictive probabilities for the linear logistic classifier.

Exercise e) Log-Likelihoods and Confusion Matrix

The final average training and test log-likelihoods are shown in Table 1. The test log-likelihood (-0.6550) is slightly lower than the training log-likelihood (-0.6161), which is expected since the model was fitted to the training data. The small gap between them confirms the absence of significant overfitting. However both values are close to $-\ln 2 \approx -0.693$, which is the log-likelihood of a random coin-flip classifier, indicating that the linear model offers only a modest improvement over random guessing on this non-linearly separable dataset.

The 2×2 confusion matrix on the test set (using threshold $\tau = 0.5$) is shown in Table 2. The classifier correctly identifies 71.88% of Class 1 ($y = 1$) points and 61.54% of Class 0 ($y = 0$) points. However, the error rates are substantial: 38.46% of true Class 0 points are misclassified as Class 1, and 28.12% of true Class 1 points are misclassified as Class 0. This confirms that the linear decision boundary is inadequate for this dataset.

Avg. Train ll	Avg. Test ll
-0.6161	-0.6550

Table 1: Average training and test log-likelihoods for the linear classifier.

		$\hat{y}$	
		0	1
y	0	0.6154	0.3846
	1	0.2812	0.7188

Table 2: Confusion matrix on the test set (fractions conditioned on true label).

Avg. Train ll	Avg. Test ll	Avg. Train ll	Avg. Test ll	Avg. Train ll	Avg. Test ll
-	-	-	-	-	-

Table 3: Results for $l = 0.01$ Table 4: Results for $l = 0.1$ Table 5: Results for $l = 1$

		$\hat{y}$	
		0	1
y	0	-	-
	1	-	-

		$\hat{y}$	
		0	1
y	0	-	-
	1	-	-

		$\hat{y}$	
		0	1
y	0	-	-
	1	-	-

Table 6: Conf. matrix $l = 0.01$.Table 7: Conf. matrix $l = 0.1$.Table 8: Conf. matrix $l = 1$.

1 Exercise f)

We now expand the inputs through a set of Gaussian radial basis functions centred on the training datapoints. We consider widths $l = \{0.01, 0.1, 1\}$ for the basis functions. We fix the learning rate parameter to be $\eta = \{\dots, \dots, \dots\}$ for each $l = \{0.01, 0.1, 1\}$, respectively. Figure ?? displays the visualisation of the contours of the resulting class predictive probabilities on top of the data for each choice of $l = \{0.01, 0.1, 1\}$.

2 Exercise g)

The final final training and test log-likelihoods per datapoint obtained for each setting of $l = \{0.01, 0.1, 1\}$ are shown in tables 3, 4 and 5. These results indicate that... The 2×2 confusion matrices for the three models trained with $l = \{0.01, 0.1, 1\}$ are show in tables 6, 7 and 8. After analysing these matrices, we can say that... When we compare these results to those obtained using the original inputs we conclude that...

3 Conclusions

1. Draw together the most important results and their consequences.
2. List any reservations or limitations.